

Prompt Engineering, Context Engineering, and Intent Engineering: A Technical Comparison for the AI Agent Stack

Executive Summary

The AI agent engineering stack is crystallizing around three distinct but layered disciplines: **prompt engineering** (optimizing single-turn instructions), **context engineering** (managing the full information environment across sessions), and **intent engineering** (aligning agent behavior to system-level goals and constraints). This report provides a precise technical comparison of each, maps them to concrete implementations like Anthropic's MCP and Claude Code's [CLAUDE.md](#), and offers an honest assessment of what's genuinely new versus repackaged.

The short answer: context engineering is a real and necessary discipline, formalized through 1,400+ research papers and endorsed by Anthropic, Karpathy, and Tobi Lütke. Intent engineering is substantively real in what it addresses — agents that optimize the wrong thing — but its intellectual lineage runs directly through OKRs, BDI agent architecture, and design intent theory. The novelty is in the application domain, not the underlying framework.[1][2]

The Three Disciplines Compared

Dimension	Prompt Engineering	Context Engineering	Intent Engineering
Core question	"How should I phrase this?"	"What does the model need to know?"	"What should this agent want?"
Problem solved	Ambiguity in single-turn LLM interactions	Information management across multi-turn agent sessions	Goal misalignment in autonomous agent behavior
Scope	Single interaction (stateless)	Session to system lifetime (stateful)	System lifetime (persistent)
What it encodes	Instructions, tone, format, constraints	Knowledge, tools, memory, retrieved data, message history	Objectives, outcomes, constraints, decision authority, stop rules
Unit of design	The prompt text	The full informational environment	The agent's mandate and boundaries
Failure mode	Model misinterprets the request	Model lacks information or drowns in irrelevant context	Agent optimizes the wrong thing or crosses boundaries
Who practices it	End users and developers	System architects and AI engineers	Product managers, engineering leaders, and governance teams

Dimension	Prompt Engineering	Context Engineering	Intent Engineering
Temporal persistence	None (one-shot)	Session-level with optional cross-session persistence	Permanent until explicitly revised

Sources:[3][4][5]

Prompt Engineering: "What to Do"

Definition

Prompt engineering is the practice of crafting precise textual instructions to elicit desired behavior from an LLM in a single inference call. It focuses on clarity of instruction, output formatting, role assignment, and techniques like few-shot prompting and chain-of-thought reasoning.[5]

Scope and Encoding

Prompt engineering operates at the level of a single interaction. It is stateless — the prompt itself is the entire system. Prompt engineering encodes the user's immediate intent through carefully structured language: instruction hierarchy, output shaping, and one-turn performance maximization. Common techniques include few-shot prompting (providing examples), chain-of-thought reasoning (asking the model to show its work), and role assignment (giving the model a persona).[3][5]

Failure Mode

When prompt engineering is done poorly or skipped, the failure mode is **ambiguity**: the model misinterprets the request, produces wrong format outputs, or makes incorrect assumptions about what's being asked. A poorly phrased prompt about "handle customer support tickets" gives the agent no guidance on priority, tone, or escalation behavior.[6]

Current Status

Prompt engineering is increasingly understood as a **subset** of context engineering rather than a standalone discipline. As Anthropic puts it, "prompt engineering refers to methods for writing and organizing LLM instructions for optimal outcomes," while context engineering encompasses "the set of strategies for curating and maintaining the optimal set of tokens during LLM inference". In mature enterprise systems, prompts are assembled, constrained, and injected as part of a controlled context strategy.[4][3]

Context Engineering: "What to Know"

Definition

Context engineering is the discipline of dynamically managing the entirety of information an LLM sees at every step of a complex task — ensuring the model has precisely the right information, at the right time, in the right format. Anthropic defines context as "the set of tokens included when sampling from a large-language model," and the engineering problem as "optimizing the utility of those tokens against the inherent constraints of LLMs".[7][4]

Scope and Encoding

Context engineering operates at the session and system level. It is stateful, managing system instructions, tools, MCP connections, external data, message history, and memory across multi-turn interactions. A July 2025 survey covering over 1,400 research papers decomposed context engineering into foundational components: context retrieval and generation, context processing, and context management — integrated through RAG, memory systems, tool-integrated reasoning, and multi-agent architectures.[1][4]

The term was popularized in mid-2025 by Shopify CEO Tobi Lütke, who wrote: "I really like the term 'context engineering'... it describes the core skill better: the art of providing all the context for the task to be plausibly solvable by the LLM." Andrej Karpathy emphatically endorsed it, describing LLMs as CPUs with context windows as RAM,

and context engineering as the operating system that loads the right data for each task.[2][8]

Key Techniques

Anthropic's engineering team outlines three primary techniques for long-horizon context management:[4]

- **Compaction:** Summarizing a conversation nearing the context window limit and reinitiating with the compressed summary. Claude Code implements this by preserving architectural decisions and unresolved bugs while discarding redundant tool outputs.
- **Structured note-taking:** Agents write persistent notes outside the context window (like a [NOTES.md](#) file), which get pulled back in at later steps. Claude playing Pokémon demonstrated this by maintaining precise tallies across thousands of game steps.
- **Sub-agent architectures:** Specialized sub-agents handle focused tasks with clean context windows, returning only condensed summaries (typically 1,000–2,000 tokens) to the orchestrating agent.

Failure Mode

When context engineering is missing, the failure mode is **retrieval failure and context pollution**: the agent either lacks critical information, retrieves wrong or stale documents, or drowns in irrelevant context that degrades performance. As context windows grow, the n^2 pairwise attention relationships mean models show reduced precision for information retrieval at longer contexts — a performance gradient, not a hard cliff.[5][4]

The AgenticSE Book's Framing

Ahmed Hassan's *Agentic Software Engineering* textbook (2026) dedicates Chapter 5 to context engineering, framing it as "managing knowledge for stochastic teammates." The core concept: "context is an interface, not a dump." The book defines practices including seeding a minimal working set, actively managing context load during execution, quarantining exploration, and transferring clean continuity across sessions.[9]

Intent Engineering: "What to Want"

Definition

Intent engineering is the practice of structuring what an AI agent should optimize for — its objectives, success criteria, constraints, decision authority, and failure modes — in a way that survives real-world edge cases where instructions run out. As the Product Compass framework states: "Intent is what determines how an agent acts when instructions run out".[6]

Scope and Encoding

Intent engineering operates at the system lifetime level. It encodes the agent's persistent mandate through a structured framework:[6]

1. **Objective:** The problem to solve and why it matters (qualitative, aspirational)
2. **Desired Outcomes:** Observable states indicating success (measurable, from user perspective)
3. **Health Metrics:** What must not degrade while pursuing outcomes (guards against Goodhart's Law)
4. **Strategic Context:** The system the agent operates within
5. **Constraints:** Both steering constraints (prompt layer) and hard constraints (enforced in orchestration)
6. **Decision Authority:** Which decisions the agent may take autonomously versus must escalate
7. **Stop Rules:** When to halt, escalate, or mark complete

Failure Mode

When intent engineering is missing, the failure mode is **misaligned optimization**: the agent pursues a technically valid but strategically wrong objective. A customer support agent told to "resolve issues faster" with no health metrics will rush responses, tanking quality. An agent told to "reduce escalations" with no constraints will handle situations it shouldn't. This is the classic Goodhart's Law problem applied to autonomous systems.[6]

Key Practitioners

The intent engineering framework was most thoroughly articulated by Pawel Huryn in Product Compass (January 2026), who explicitly maps it to OKRs as defined by Christina Wodtke and empowered team objectives as described by Marty Cagan. Onkar Gosavi has written about intent as a "structured contract" defining why the agent exists, what success looks like, and what boundaries must never be crossed. The *Agentic Software Engineering* textbook frames the same concept as "Mission Engineering" — Chapter 4 is titled "Making intent explicit and verifiable" and introduces the "Mission Brief" as a contract for autonomy, with practices including intent alignment, property-controlled acceptance, and autonomy envelope definition.[10][9][6]

Concrete Example: The Same Agent Task at Each Level

Task: Build an AI agent that handles Tier-1 customer support for a SaaS product.

Prompt Engineering Layer

You are a helpful customer support agent for Acme SaaS. Answer questions about billing, account settings, and basic troubleshooting. Be friendly and concise. If you don't know the answer, say so. Format responses in plain text.

This handles a single turn well but provides no session memory, no access to customer data, and no guidance on what the agent should optimize for over time.

Context Engineering Layer

The context engineering layer builds the information architecture around the prompt:[7][4]

- **System prompt** with role, behavioral guidelines, and tool guidance

- **MCP servers** connecting to the billing API, knowledge base, and ticket system
- **Retrieved context:** customer's subscription tier, recent tickets, and account status pulled via RAG or just-in-time tool calls
- **Message history** management: compaction after 20 turns, preserving unresolved issues
- **CLAUDE.md-style persistent memory:** product-specific terminology, common edge cases, escalation paths
- **Tool definitions:** lookup_account, check_billing, create_ticket, escalate_to_human

Now the agent knows what it needs to know at each step. But it still has no explicit definition of what success looks like or where its authority ends.

Intent Engineering Layer

The intent layer defines the agent's mandate:[6]

Objective: Help customers resolve Tier-1 issues quickly so they can get back to work, without creating more frustration than they started with.

Desired Outcomes:

- Customer confirms their issue is resolved
- No follow-up ticket on same topic within 24h
- Customer rates interaction as helpful

Health Metrics:

- CSAT stays above 4.2 — be more conservative if trending down
- Repeat contact rate stays below 12%
- Escalation quality score stays within band

Constraints (Hard):

- Never process refunds over \$500 without human approval
- Never access or disclose another customer's data
- Never promise features not on the public roadmap

Decision Authority:

- Autonomous: password resets, billing explanations, known bug workarounds
- Escalate: account cancellations, security incidents, billing disputes >\$500

Stop Rules:

- After 3 failed resolution attempts, escalate to human
- If customer expresses anger twice, offer human handoff

With all three layers in place, the agent knows what to do (prompt), what to know (context), and what to want (intent).

Anthropic's MCP as Context Engineering Infrastructure

The Model Context Protocol (MCP), released by Anthropic in November 2024, is the infrastructure layer that makes context engineering scalable. MCP is an open standard that provides a universal protocol for connecting AI systems to external data sources, replacing the N×M integration problem — where connecting N models to M data sources required bespoke connectors — with a standardized interface.[11][12]

MCP follows a client-server architecture using JSON-RPC 2.0 messages and provides three core primitives: **tools** (actions the agent can take), **resources** (data the agent can access), and **prompts** (reusable prompt templates). With over 8 million weekly SDK downloads as of mid-2025, MCP has become the de facto standard for AI-to-tool communication.[13][12]

The relationship to context engineering is direct: MCP is the plumbing that delivers context to agents at runtime. As Anthropic's engineering blog explains, agents built with "just-in-time" context strategies maintain lightweight identifiers (file paths, stored queries, web links) and use MCP tools to dynamically load data into context at runtime — rather than pre-processing all relevant data up front. This approach mirrors how Claude Code uses MCP connections alongside `glob` and `grep` to navigate its environment.[4]

The shift from RAG-only architectures to MCP-enabled ones represents the technical evolution that made context engineering a

distinct discipline. Where RAG optimized retrieval for a single query, MCP enables the continuous, bidirectional flow of context that multi-turn agents require.[14]

CLAUDE.md as a Persistent Context/Intent Layer

Claude Code's [CLAUDE.md](#) file is a concrete artifact that bridges context engineering and intent engineering. The file acts as persistent memory, loaded into Claude Code's context at every session start. It operates at three hierarchical levels:[15][16][17]

- **Global** (~/.claude/CLAUDE.md): Cross-project user preferences and patterns
- **Project** (repository root CLAUDE.md): Shared team context, checked into version control
- **Local** (CLAUDE.local.md): Private developer-specific notes, excluded from version control

The file encodes both **context** (project structure, code style, build commands, architectural decisions, domain-specific terminology) and **intent** (coding standards to follow, patterns to avoid, escalation behaviors, testing requirements). This dual role makes [CLAUDE.md](#) a practical example of how the three disciplines collapse in implementation: the file *is* a prompt (instructions), *carries* context (project knowledge), and *embodies* intent (what the agent should optimize for within this codebase).[17][15]

The emerging [AGENTS.md](#) standard attempts to consolidate tool-specific formats ([CLAUDE.md](#), `.cursorrules`, etc.) into a single specification, though adoption analysis of 466 open-source projects found "no established content structure yet" with significant variation in how context is provided — descriptive, prescriptive, prohibitive, explanatory, and conditional.[18]

Amazon's Evaluation Framework for Agent Intent Detection

Amazon's approach to evaluating AI agents provides the most comprehensive production-grade framework for measuring whether agents correctly understand intent. Since 2025, thousands of agents have been built across Amazon organizations, requiring a fundamental shift from single-model benchmarks to system-level evaluation.[19]

Three-Layer Evaluation Architecture

Amazon's framework operates across three layers:[19]

Layer	What It Evaluates	Key Metrics
Bottom	Foundation model performance	Model benchmarks, latency
Middle	Agent components (intent detection, reasoning, tool use, memory)	Intent correctness, tool selection accuracy, CoT grounding
Upper	End-to-end task completion and business outcomes	Goal success, safety, customer experience, cost

Intent Detection Methodology

For their customer service AI agent, Amazon built an **LLM simulator** using LLM-driven virtual customer personas to simulate diverse user scenarios and interactions. The evaluation centers on whether the orchestration agent correctly detects customer intent and routes to the appropriate specialized resolver (subagent).[19]

The methodology uses simulation datasets of user query and ground truth intent pairs from anonymized historical interactions. Key metrics include intent correctness (comparing generated intents to ground truth), task completion (final response quality and resolution), topic adherence classification, and topic adherence

refusal. For tool use, Amazon targets **95% tool selection accuracy** and **98% parameter extraction accuracy**. [20][19]

Amazon also published research on intent detection using LLMs, finding that a hybrid system combining contrastively fine-tuned sentence transformers with LLMs (via uncertainty-based routing) achieves "within 2% of native LLM accuracy with 50% less latency". [21]

Is Intent Engineering Genuinely New?

This is the central question, and the honest answer is: **the problem is real, the framework is not new, but the application domain creates genuine new challenges.**

The Intellectual Lineage

Intent engineering maps directly to at least three established disciplines:

OKRs and Empowered Teams: The Product Compass framework explicitly acknowledges that its intent structure "shows up in OKRs, as defined by Christina Wodtke, and in how Marty Cagan defines empowered team objectives". The objective/outcome/health metric structure is nearly identical to a well-written OKR set with guardrail metrics. [6]

BDI Agent Architecture: The Belief-Desire-Intention software model, developed from Michael Bratman's theory of human practical reasoning, has been the standard framework for programming intelligent agents since the 1990s. BDI separates beliefs (world knowledge — analogous to context), desires (goals — analogous to objectives), and intentions (committed plans — analogous to decision authority and stop rules). The parallel is not superficial; it's structural. [22]

Design Intent in Engineering: Manufacturing and systems engineering have long used "design intent" — the documented rationale for why something was designed a certain way — as a governance mechanism. The concept of encoding constraints,

tolerances, and acceptance criteria for non-deterministic processes is standard in civil, aerospace, and reliability engineering.

What's Genuinely New

The WhatIsAFramework analysis puts it plainly: "This is not new territory". But three factors make the application domain meaningfully different:[23]

1. **Agents can't ask clarifying questions in the same way humans can.** A human employee given vague OKRs will use social context, organizational norms, and informal conversations to fill gaps. An agent given vague intent will silently optimize the wrong thing. This raises the stakes on specification completeness dramatically.
2. **Hard constraints must be enforced architecturally, not culturally.** In human organizations, many constraints are enforced through social norms, peer review, and management oversight. Intent engineering for agents requires splitting constraints into steering constraints (encoded in prompts) and hard constraints (enforced in orchestration code, not prompts). This is a genuine engineering discipline.[6]
3. **The speed and scale of agent execution compresses feedback loops.** The *Agentic Software Engineering* textbook frames this as a bottleneck flip: when output explodes 100x, "the scarce resource is no longer typing; it is attention. Humans must own intent and risk boundaries". An agent acting on misaligned intent can do damage at machine speed across multiple systems simultaneously.[9]

The Honest Assessment

Intent engineering is **existing concepts (OKRs, BDI, design intent) rigorously applied to a new class of autonomous actor that cannot rely on implicit organizational context**. The novelty is real but incremental. The term risks being dismissed as marketing if its practitioners don't acknowledge the lineage — and risks being underestimated if critics don't acknowledge that applying these frameworks to stochastic, autonomous agents requires genuine engineering discipline that doesn't yet exist in most organizations.

The most useful framing comes from the *Agentic Software Engineering* textbook, which doesn't use the term "intent engineering" at all, instead calling it "Mission Engineering" and treating it as one of several engineering disciplines (alongside context engineering, trust engineering, and coordination engineering) required to produce trustworthy software from stochastic contributors.[9]

The Relationship to Agentic Software Engineering

The open-access textbook *Agentic Software Engineering* by Ahmed Hassan (agenticse-book.github.io, 2026) provides the most comprehensive systems-level framing of how these disciplines interrelate.[24][9]

Hassan's central thesis: "Agentic Software Engineering is the discipline of producing high-quality, reliable, trustworthy software from stochastic contributors, both AI and human, by making the full SE system ready: people, process, tools, and artifacts". The book structures its assurance engineering across four disciplines that map cleanly to the prompt/context/intent taxonomy:[9]

Book Discipline	Maps To	Chapter
Mission Engineering	Intent Engineering	Ch. 4: "Making intent explicit and verifiable"
Context Engineering	Context Engineering	Ch. 5: "Managing knowledge for stochastic teammates"
Trust Engineering	Cross-cutting (verification, evidence)	Ch. 7
Coordination Engineering	Multi-agent orchestration	Ch. 6

The book introduces the **Mission Brief** as the concrete artifact for intent — a structured contract that defines the objective, acceptance properties, autonomy envelope, and escalation paths. Prompt engineering is subsumed within these broader disciplines rather than treated as a standalone concern.[9]

Notably, the book frames the transition as SE 1.0 (human-driven, classical tools) → SE 2.0 (copilots, still human-in-loop) → SE 3.0 (agentic, where output explodes and "the scarce resource is no longer typing; it is attention"). In SE 3.0, intent and context engineering become the primary control surfaces — not as new ideas, but as existing engineering practices elevated to first-class architectural concerns.[9]

Synthesis: Three Layers, One Stack

The three disciplines are not alternatives but **layers in a stack**, each solving a different class of failure:

Layer	Solves	Without It	Implementation Artifact
Prompt Engineering	"Model doesn't understand the instruction"	Ambiguous, misformatted outputs	System prompt, few-shot examples
Context Engineering	"Model doesn't have the right information"	Hallucination, stale data, context overflow	MCP servers, RAG pipelines, CLAUDE.md , compaction strategies
Intent Engineering	"Model optimizes the wrong thing"	Misaligned autonomous behavior at scale	Mission briefs, OKR-like objective structures, hard constraint enforcement

Prompt engineering is becoming a subset of context engineering. Context engineering is the bridge between model capability and real-world knowledge. Intent engineering is the governance layer that determines whether the agent's capabilities and knowledge are directed toward the right outcomes.[3][4]

For practitioners building AI agents today, the practical implication is clear: start with intent (what should the agent optimize?), build the context architecture (what does the agent need to know?), and let prompt engineering handle the remaining single-turn optimization within that governed framework. The teams that will build reliable agents are not the ones with the best prompts — they are the ones with the best methodology for defining agent intent and the engineering discipline to enforce it.[23][9]

References

1. [A Survey of Context Engineering for Large Language Models](#) - The performance of Large Language Models (LLMs) is fundamentally determined by the contextual inform...
2. [Context Engineering: Bringing Engineering Discipline to Prompts](#) - The phrase was popularized by developers like Shopify's CEO Tobi Lütke and AI leader Andrej Karpathy...
3. [Context Engineering vs Prompt Engineering - Blog about AI ...](#) - Context engineering focuses on designing, building, and managing the complete informational system a...
4. [Effective context engineering for AI agents - Anthropic](#) - In this post, we'll explore the emerging art of context engineering and offer a refined mental model...
5. [Context engineering vs. prompt engineering - Elasticsearch Labs](#) - Prompt engineering focuses on how you communicate with the model. Context engineering focuses on wha...
6. [The Intent Engineering Framework for AI Agents](#) - The Intent Engineering Framework shows how to structure objectives, outcomes, health metrics, constr...
7. [Why 'Context Engineering' is the New Frontier for AI Agents](#) - 2026 Marketer's Guide to AI Agents for Marketing Operations. LLM ... What's shaping AI products, age...

8. [What is Context Engineering, Anyway? - Zep](#) - Tobi Lütke, CEO of Shopify, publicly endorsed the terminology on X (formerly Twitter), emphasizing t...
9. [\[PDF\] Agentic Software Engineering](#) - So this book is about Agentic Software Engineering, not agentic coding. It is about how to run softw...
10. [Intent Engineering for AI Agents: A Structured Approach to Success](#) - A DeepWork Agent is defined by its execution discipline: explicit constraints, typed outputs, tool-m...
11. [Model Context Protocol - Open standard for connecting AI assistants](#) - The Model Context Protocol is an open standard that enables developers to build secure, two-way conn...
12. [The Model Context Protocol: Getting beneath the hype](#) - The Model Context Protocol is an open standard that defines and stabilizes the way developers build ...
13. [Model Context Protocol \(MCP\) at First Glance: Studying the Security and Maintainability of MCP Servers](#) - Although Foundation Models (FMs), such as GPT-4, are increasingly used in domains like finance and s...
14. [AI battle scars and ROI: Why context engineering is key for agents](#) - ... 2025, everyone was talking about agents and agentic architectures; 2026? I can guarantee you it'...
15. [Creating the Perfect CLAUDE.md for Claude Code - Dometrain](#) - The [CLAUDE.md](#) file acts as persistent memory for Claude Code agents. ... Every agent you start loads...
16. [The Architecture of Persistent Memory for Claude Code - Dev.to](#) - Claude Code already reads a file called [CLAUDE.md](#) on every session start. This is baked into how Cla...
17. [Claude Code's Memory: Working with AI in Large Codebases](#) - The Claude Code Memory System: Your Persistent Context Architect. TL ... [CLAUDE.md](#) and [CLAUDE.local](#)...
18. [Context Engineering for AI Agents in Open-Source Software](#) - GenAI-based coding assistants have disrupted software development. The next generation of these tool...
19. [Evaluating AI agents: Real-world lessons from building agentic ...](#) - Evaluating user intent detection in the Amazon customer service AI agent ... The stakes are high whe...
20. [AI agents in enterprises: Best practices with Amazon Bedrock ...](#) - When you detect drift, such as accuracy dropping from 92% to

- 84% over two weeks, investigate and add...
21. [Intent detection in the age of LLMs - Amazon Science](#) - We also introduce a two-step approach utilizing internal LLM representations, demonstrating empirica...
 22. [Belief–desire–intention software model - Wikipedia](#) - BDI agents. A BDI agent is a particular type of bounded rational software agent, imbued with particu...
 23. [Intent Engineering for AI Agents: The Framework Methodology ...](#) - In January 2026, Product Compass published a detailed breakdown of "intent engineering" — the discipl...
 24. [Agentic Software Engineering - The Future of Code](#) - A comprehensive guide to Agentic Software Engineering. Learn how AI agents are transforming the soft...